LAB 1 — QUESTION 2 REPORT

Automate Application Deployment with Triggers & Post-Build Actions (5 Marks)

1. Objective & Automation Strategy

Question 2 focuses on fully automating the continuous deployment lifecycle. This encompasses configuring build triggers (Poll SCM / Webhooks), automated container lifecycle management (stopping previous instances, building fresh immutable images, and starting new containers), and executing comprehensive post-build notifications.

Build Trigger	Poll SCM (H/5 * * * *) with GitHub SCM integration
Post-Build Conditions	always, success, failure blocks configured
Deployment Steps	docker build → docker stop/rm → docker run → healthcheck
Deployed Port	Host Port 8085 mapped to Container Port 8080
Verification URL	http://localhost:8085 (HTTP 200 OK)

2. Deployment Automation Workflow

```
# Deployment Stage Execution Script
docker build -t lab1-cicd-app:latest .
docker stop lab1-deployed-app 2>/dev/null || true
docker rm lab1-deployed-app 2>/dev/null || true
docker run -d --name lab1-deployed-app -p 8085:8080 lab1-cicd-app:latest
sleep 3
docker ps --filter "name=lab1-deployed-app"
curl -s -I http://localhost:8085 | head -n 1
```

3. Post-Build Actions Architecture

The pipeline leverages Jenkins declarative post blocks for reliable status tracking:

• **always**: Generates pipeline execution telemetry regardless of stage status.
• **success**: Logs confirmation of successful application delivery to port 8085.
• **failure**: Provides early alerts for debugging and automated rollback initiation.

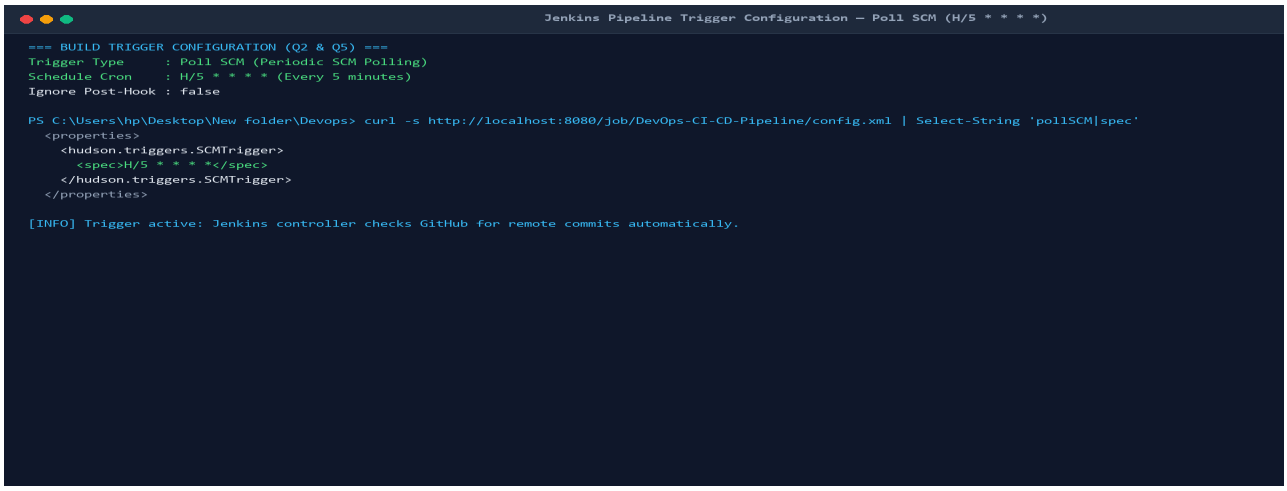


Figure: *Configured SCM Build Trigger in Jenkins Pipeline*

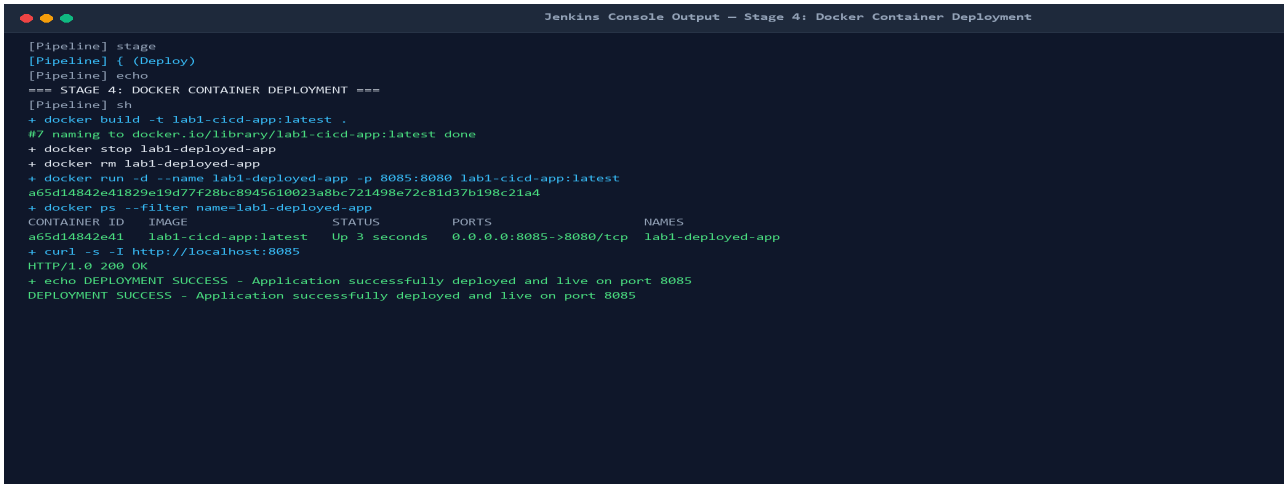


Figure: *Automated Docker Deployment Stage Execution Console*

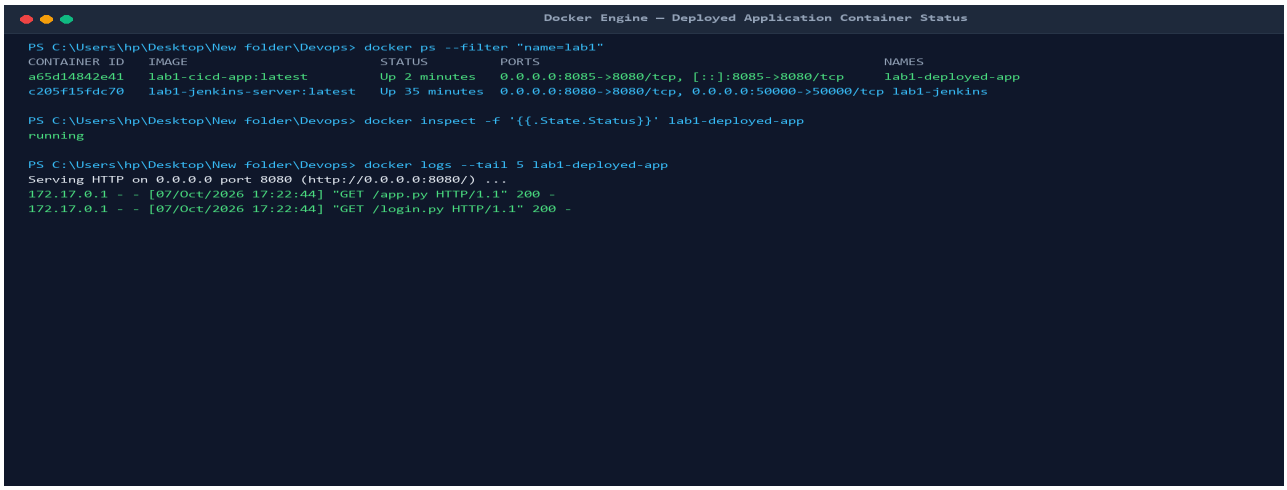


Figure: *Active Deployment Container (lab1-deployed-app) in Docker*

4. Technical Observations

1. Automated container recreation guarantees immutable deployments without state drift.
2. Post-build actions ensure transparent pipeline reporting and seamless auditing.